



WYDZIAŁ
ELEKTROTECHNIKI
I INFORMATYKI
POLITECHNIKI RZESZOWSKIEJ



WYDZIAŁ
MATEMATYKI
I FIZYKI STOSOWANEJ
POLITECHNIKI RZESZOWSKIEJ

Kierunek: **Inżynieria i analiza danych**

Sprawozdanie z projektu

z przedmiotu: *Bezpieczeństwo i ochrona danych*

Analiza bezpieczeństwa aplikacji "Travelly"

Autorzy:

Agnieszka Słapińska, Magdalena
Stanek, Natalia Szczupak

Prowadzący:

dr inż. Mirosław Mazurek

Rzeszów, 2025

Spis treści

1	Wstęp	1
2	Bezpieczne przechowywanie haseł użytkowników	2
2.1	Szczegółowy opis algorytmu PBKDF2 i wektorów ataku	2
2.2	Weryfikacja implementacji	3
2.3	Wymuszanie złożoności haseł (Settings.py)	4
3	Ochrona przed wstrzykiwaniem kodu SQL	7
3.1	Istota zagrożenia	7
3.2	Mechanizmy wbudowane i walidacja	7
3.2.1	Weryfikacja skuteczności	8
4	Ochrona przed botami (CAPTCHA)	10
4.1	Implementacja techniczna (django-simple-captcha)	10
4.2	Proces walidacji po stronie serwera	11
4.3	Interfejs użytkownika	11
4.4	Weryfikacja adresu e-mail (Double Opt-In)	12
5	Ochrona przed atakami XSS (Cross-Site Scripting)	13
5.1	Mechanizm auto-escape w Django	13
5.2	Test podatności (Weryfikacja)	14
6	Bezpieczeństwo transmisji danych (HTTP vs HTTPS)	15
6.1	Analiza porównawcza protokołów	15
6.2	Weryfikacja szyfrowania (Tunelowanie SSL)	15
6.3	Konfiguracja Environment Hardening	16
7	Zarządzanie Sesją i Ochrona CSRF	17
7.1	Czym jest atak CSRF?	17
7.2	Ochrona CSRF w aplikacji	17
7.3	Bezpieczeństwo ciasteczek (HttpOnly)	18
8	Zabezpieczenia nagłówków HTTP i Infrastruktury	19
8.1	Ochrona przed Clickjackingiem (Niewidzialne ramki)	19
8.2	Ochrona przed podszywaniem się (Host Header Injection)	19
9	Aktywna ochrona przed atakami Brute-Force	20
9.1	Instalacja i podłączenie 'ochroniarza'	20
9.2	Konfiguracja polityki bezpieczeństwa	21
9.3	Weryfikacja skuteczności (Symulacja ataku)	22
10	Monitorowanie i Rejestrowanie Zdarzeń (Audyt)	23
10.1	Logi systemowe i historia zmian	23
11	Szczegółowa analiza konfiguracji (settings.py)	24
11.1	Sekcja 1: Podstawowe ustawienia środowiska	24
11.2	Sekcja 2: Aplikacje zewnętrzne i Middleware	25
11.3	Sekcja 3: Baza danych i walidatory (cz. 1)	26

11.4	Sekcja 4: Zaawansowana walidacja haseł i lokalizacja	26
11.5	Sekcja 5: Pliki statyczne i Backendy Uwierzytelniania	27
11.6	Sekcja 6: Ochrona Brute-Force i Environment Hardening	28
12	Analiza Ryzyka i Podsumowanie	30
12.1	Macierz Analizy Ryzyka	30
12.2	Wnioski	30

Spis rysunków

1	Lista użytkowników w panelu administracyjnym Django.	3
2	Szczegóły konta. Widoczny format zapisu algorytmu PBKDF2, sól oraz milion iteracji.	4
3	Konfiguracja walidatorów haseł w settings.py.	4
4	Fragment kodu weryfikujący siłę hasła (wymóg znaku specjalnego).	5
5	Odrzucenie słabego hasła przez system.	6
6	Kod funkcji walidującej niedozwolone znaki (czarna lista znaków).	8
7	Implementacja walidatora w klasie formularza.	8
8	Próba wstrzyknięcia kodu SQL w formularzu.	9
9	Zablokowanie ataku przez system walidacji.	9
10	Implementacja pola CaptchaField w klasie formularza.	10
11	Formularz rejestracji z widocznym, zaszumionym obrazem CAPTCHA.	11
12	Komunikat o konieczności weryfikacji po wstępnej rejestracji.	12
13	Wiadomość e-mail z kodem weryfikacyjnym generowanym losowo.	12
14	Wprowadzanie kodu aktywacyjnego.	12
15	Pomyślna aktywacja konta w systemie.	12
16	Wynik próby ataku XSS – kod został zneutralizowany i wyświetlony jako tekst.	14
17	Aplikacja działająca pod protokołem HTTPS (widoczna kłódka i komunikat "Połączenie jest bezpieczne").	16
18	Aktywacja ochrony CSRF w settings.py (CsrfViewMiddleware).	17
19	Ukryty token CSRF widoczny w źródle strony HTML.	18
20	Analiza ciasteczek w narzędziach deweloperskich. Widoczna flaga HttpOnly.	18
21	Konfiguracja dozwolonych hostów w pliku settings.py – serwer obsługuje tylko te adresy.	19
22	Rejestracja modułu axes w konfiguracji projektu.	20
23	Aktywacja Middleware, który monitoruje ruch przychodzący.	21
24	Definicja limitów prób logowania w settings.py.	21
25	Kod szablonu strony wyświetlanej zablokowanemu użytkownikowi.	22
26	Efekt działania zabezpieczenia – konto zablokowane po przekroczeniu limitu.	22
27	Dziennik zdarzeń (historia operacji) w panelu administracyjnym.	23
28	Zmienne środowiskowe, tryb debugowania i zainstalowane aplikacje.	24
29	Rejestracja bibliotek bezpieczeństwa i konfiguracja Middleware.	25
30	Konfiguracja bazy danych SQLite i pierwszy walidator haseł.	26
31	Lista walidatorów haseł, w tym autorski ComplexityValidator.	27
32	Konfiguracja backendów uwierzytelniania dla modułu Axes.	28
33	Polityka blokad (Axes) i utwardzanie środowiska produkcyjnego.	29

1 Wstęp

Celem niniejszego projektu jest implementacja oraz weryfikacja mechanizmów bezpieczeństwa w aplikacji internetowej "Travelly". Zadaniem opracowania jest przedstawienie sposobów, w jakie system chroni dane wrażliwe oraz zapobiega nieautoryzowanemu dostępowi. Aplikacja została zbudowana w oparciu o framework Django, który posiada szereg wbudowanych zabezpieczeń, jednak wymaga odpowiedniej konfiguracji w pliku `settings.py` oraz implementacji dodatkowych modułów.

W sprawozdaniu omówiono kluczowe filary bezpieczeństwa:

- **Mechanizmy wbudowane:** Bezpieczne przechowywanie haseł.
- **Ochrona danych wejściowych:** Walidacja SQL Injection oraz XSS.
- **Zabezpieczenia dodatkowe:** CAPTCHA, weryfikacja tożsamości.
- **Polityka bezpieczeństwa:** Ochrona sesji, CSRF, Clickjacking.
- **Ochrona aktywna:** Blokada ataków Brute-Force.

2 Bezpieczne przechowywanie haseł użytkowników

Jednym z fundamentalnych wymogów bezpieczeństwa jest ochrona danych uwierzytelniających. W aplikacji "Travelly" hasła nigdy nie są przechowywane jawnym tekstem, lecz jako bezpieczne skróty (hashe).

2.1 Szczegółowy opis algorytmu PBKDF2 i wektorów ataku

Aplikacja wykorzystuje standard przemysłowy – algorytm **PBKDF2** (Password-Based Key Derivation Function 2) z funkcją skrótu **SHA256**. Wybór ten podyktowany jest koniecznością ochrony przed specyficznymi technikami łamania haseł.

Proces generowania bezpiecznego hasła opisuje wzór:

$$DK = PBKDF2(PRF, Password, Salt, c, dkLen)$$

Zastosowanie tego algorytmu stanowi bezpośrednią odpowiedź na następujące zagrożenia:

- **Ochrona przed tablicami tęczowymi (Rainbow Tables):** Atak ten polega na wykorzystaniu gotowych, wstępnie obliczonych gigantycznych baz danych zawierających popularne hasła i odpowiadające im skróty (hashe). Bez użycia soli, hasło "admin1" zawsze miałoby ten sam hash w każdej bazie danych. **Rozwiązanie:** Mechanizm PBKDF2 dodaje do hasła losową **sól (Salt)** – unikalny ciąg znaków generowany osobno dla każdego użytkownika. Sprawia to, że hash hasła "admin1" dla użytkownika A jest zupełnie inny niż dla użytkownika B. Czyni to tablice tęczowe bezużytecznymi, ponieważ atakujący musiałby generować nową tablicę dla każdej unikalnej soli.
- **Ochrona przed atakami słownikowymi i Brute-Force:** Ataki te polegają na sprawdzaniu milionów kombinacji słów lub znaków na sekundę. **Rozwiązanie:** Zastosowanie parametru **iteracji (c)**. W naszym systemie wartość ta wynosi **1 000 000** (jeden milion). Oznacza to, że funkcja skrótu jest wykonywana milion razy w pętli. Dla użytkownika logującego się raz, opóźnienie rzędu ułamka sekundy jest niezauważalne. Dla atakującego, który chce sprawdzić miliardy haseł, narzut ten staje się barierą nie do pokonania, drastycznie zwiększając koszt obliczeniowy i czas potrzebny na złamanie hasła (tzw. *Key Stretching*).

Analiza elementów składowych (w odniesieniu do zrzutu ekranu):

- **DK (Derived Key):** Wynikowy ciąg bitów (hash) widoczny w polu **hash**. To on jest zapisany w bazie.

- **PRF:** Funkcja pseudolosowa, w tym przypadku pbkdf2_sha256.
- **Salt:** Unikalna sól, widoczna jako h5S2MS. . . .
- **c:** Liczba iteracji ustawiona na 1 000 000.
- **dkLen (Derived Key Length):** Parametr określający stałą długość wynikowego hasha (niezależnie od długości hasła wpisanego przez użytkownika). Dzięki niemu każdy skrót w bazie danych zajmuje dokładnie tyle samo miejsca, co uniemożliwia atakującemu odgadnięcie długości oryginalnego hasła na podstawie długości zapisanego szyfrogramu.
- **Password:** Hasło użytkownika – jest jedynym elementem wzoru, który **nie znajduje się** w bazie danych (co potwierdza komunikat na zrzucie ekranu). Służy ono jedynie jako dane wejściowe w momencie logowania; jeśli po przetworzeniu z solą i iteracjami wynik zgadza się z **DK**, użytkownik zostaje wpuszczony.

2.2 Weryfikacja implementacji

Aby potwierdzić działanie mechanizmu, przeprowadzono analizę danych w panelu administracyjnym systemu (Rysunek 1). Panel ten odzwierciedla stan bazy danych.

NAZWA UŻYTKOWNIKA	ADRES E-MAIL	IMIE	NAZWISKO	W ZESPOLE
176859	-	-	-	W ZESPOLE
1768599	-	-	-	W ZESPOLE
17685999	agnieszka018@gmail.com	-	-	W ZESPOLE
admin	admin@example.com	-	-	W ZESPOLE

Rysunek 1: Lista użytkowników w panelu administracyjnym Django.

Po przejściu do szczegółów konkretnego konta (Rysunek 2), system informuje, że hasło nie jest przechowywane w formie jawnej.

Rysunek 2: Szczegóły konta. Widoczny format zapisu algorytmu PBKDF2, sól oraz milion iteracji.

2.3 Wymuszanie złożoności haseł (Settings.py)

Centralnym miejscem konfiguracji bezpieczeństwa w Django jest plik `settings.py`. To tutaj, w sekcji `AUTH_PASSWORD_VALIDATORS`, zdefiniowano reguły, które musi spełnić hasło. Obejmują one: minimalną długość, zakaz używania haseł podobnych do loginu oraz wymóg stosowania znaków o różnym typie.

```

AUTH_PASSWORD_VALIDATORS = [
    {"NAME": "django.contrib.auth.password_validation.UserAttributeSimilarityValidator"},
    {"NAME": "django.contrib.auth.password_validation.MinimumLengthValidator",
     "OPTIONS": {"min_length": 10}},
    {"NAME": "django.contrib.auth.password_validation.CommonPasswordValidator"},
    {"NAME": "django.contrib.auth.password_validation.NumericPasswordValidator"},
    {"NAME": "trips.validators.ComplexityValidator"},
]

```

Rysunek 3: Konfiguracja walidatorów haseł w settings.py.

Dodatkowo w formularzu zaimplementowano własną metodę walidacji (Rysunek 4), która za pomocą wyrażeń regularnych (Regex) sprawdza obecność znaków specjalnych.


```

if 'password1' in self.fields:
    css = self.fields['password1'].widget.attrs.get("class", "")
    self.fields['password1'].widget.attrs["class"] = (css + " form-control").strip()
    self.fields["password1"].help_text = (
        "Min. 8 znaków, nie może być zbyt podobne do danych konta. "
        "Musí zawierać co najmniej jeden znak specjalny."
    )
if 'password2' in self.fields:
    css = self.fields['password2'].widget.attrs.get("class", "")
    self.fields['password2'].widget.attrs["class"] = (css + " form-control").strip()

def clean_password2(self):
    pwd1 = self.cleaned_data.get("password1")
    pwd2 = self.cleaned_data.get("password2")

    if pwd1 and pwd2 and pwd1 != pwd2:
        raise forms.ValidationError("Hasła muszą być identyczne.")

    if pwd1 and not re.search(r"^[A-Za-z0-9]", pwd1):
        raise forms.ValidationError("Hasło musi zawierać co najmniej jeden znak specjalny (np. ! @ # $ % ^ & *).")

    return pwd2

```

Rysunek 4: Fragment kodu weryfikujący siłę hasła (wymóg znaku specjalnego).

Próba rejestracji z użyciem słabego hasła kończy się blokadą i wyświetleniem komunikatu (Rysunek 5).

Rejestracja

Nazwa użytkownika

E-mail

Hasło

Min. 8 znaków, nie może być zbyt podobne do danych konta. Musi zawierać co najmniej jeden znak specjalny.

Powtórz hasło

- Hasło musi zawierać co najmniej jeden znak specjalny (np. ! @ # \$ % ^ & *).

Weryfikacja



[Utwórz konto](#)

Masz już konto? [Zaloguj się](#)

Rysunek 5: Odrzucenie słabego hasła przez system.

3 Ochrona przed wstrzykiwaniem kodu SQL

Atak **SQL Injection (SQLi)** jest jedną z najstarszych i najgroźniejszych podatności aplikacji internetowych. Polega on na nieautoryzowanym wstrzyknięciu fragmentów zapytań SQL do danych wejściowych aplikacji (np. formularzy logowania, parametrów URL).

3.1 Istota zagrożenia

Problem pojawia się w momencie, gdy aplikacja tworzy zapytania do bazy danych poprzez proste "sklejanie" ciągów znaków (konkatenację), nie weryfikując wcześniej danych otrzymanych od użytkownika.

Przykładowy scenariusz ataku wygląda następująco:

1. Aplikacja oczekuje nazwy użytkownika, aby wstawić ją do zapytania:

```
SELECT * FROM users WHERE username = '$user_input';
```

2. Atakujący w polu loginu wpisuje frazę: ' OR '1'='1

3. Wynikowe zapytanie wysłane do bazy wygląda tak:

```
SELECT * FROM users WHERE username = '' OR '1'='1';
```

Ponieważ warunek '1'='1' jest zawsze prawdziwy (tautologia), baza danych zwróci rekordy wszystkich użytkowników, pozwalając na zalogowanie się bez hasła (często jako administrator). W bardziej zaawansowanych scenariuszach (np. *UNION-Based SQLi*), atakujący może odczytać całą zawartość bazy, a nawet usunąć tabele.

3.2 Mechanizmy wbudowane i walidacja

Framework Django zapewnia domyślną, potężną ochronę przed SQL Injection poprzez warstwę abstrakcji ORM (Object-Relational Mapping). Zapytania nie są budowane poprzez sklejanie ciągów znaków, lecz parametryzowane.

Dodatkowo, dla zwiększenia bezpieczeństwa warstwy wejściowej (tzw. *Defense in Depth*), zaimplementowano autorski walidator blokujący znaki specjalne często używane w atakach SQL, takie jak apostrofy, średniki czy myślniki (Rysunek 6).

```
# === OCHRONA PRZED SQL INJECTION ===
def check_for_sql_injection_chars(value):
    """
    Sprawdza, czy tekst nie zawiera znaków typowych dla
    ataków SQL Injection, takich jak średnik (;).
    """
    # Lista znaków, których nie chcemy
    dangerous_chars = [';', '--', '/*']

    for char in dangerous_chars:
        if char in value:
            raise ValidationError(
                f"Wykryto niedozwolony znak: '{char}'. Ze względów bezpieczeństwa usuń go."
            )
```

Rysunek 6: Kod funkcji walidującej niedozwolone znaki (czarna lista znaków).

Walidator ten podpięto do pól formularza kontaktowego w klasie formularza (Rysunek 7), co wymusza sprawdzenie danych zanim trafią one do dalszego przetwarzania.

```
class ContactForm(forms.Form):
    name = forms.CharField(
        label="Imię i nazwisko",
        max_length=120,
        validators=[check_for_sql_injection_chars]
    )
    email = forms.EmailField(label="E-mail")
    message = forms.CharField(
        label="Wiadomość",
        widget=forms.Textarea,
        max_length=4000,
        validators=[check_for_sql_injection_chars]
    )
```

Rysunek 7: Implementacja walidatora w klasie formularza.

3.2.1 Weryfikacja skuteczności

Przeprowadzono próbę ataku poprzez wpisanie klasycznej komendy SQL mającej na celu obejście logowania lub wywołanie błędu składni (Rysunek 8).

Skontaktuj się z nami

Masz pytania? Chcesz zaplanować podróż marzeń? Jesteśmy tu, aby pomóc!

Napisz do nas

Imię i nazwisko

Agnieszka Stąpowska

E-mail

176859@gmail.com

Wiadomość

To jest test: DROP TABLE users

Wyślij wiadomość

Informacje kontaktowe

Adres:
Biuro Travelly
ul. Wakacyjna 1, 35-000 Rzeszów

Telefon:
+48 123 456 789

E-mail:
kontakt@travelly.local

Rysunek 8: Próba wstrzyknięcia kodu SQL w formularzu.

System poprawnie zidentyfikował niedozwolone znaki i zablokował wysłanie formularza, wyświetlając stosowny komunikat błędu (Rysunek 9).

TRAVELLY [O nas](#) [Usługi](#) [Oferty](#) [Kontakt](#) Cześć, 176859! [Ulubione](#) [Wyloguj](#) [Admin](#)

Popraw błędy w formularzu i spróbuj ponownie.

Rysunek 9: Zablokowanie ataku przez system walidacji.

4 Ochrona przed botami (CAPTCHA)

W celu zabezpieczenia formularza rejestracji przed automatycznymi skryptami (botami), które mogłyby masowo tworzyć fałszywe konta lub obciążać serwer (ataki DoS), wdrożono mechanizm **CAPTCHA** (ang. *Completely Automated Public Turing test to tell Computers and Humans Apart*).

4.1 Implementacja techniczna (django-simple-captcha)

W projekcie wykorzystano bibliotekę `django-simple-captcha`, która integruje się z systemem formularzy Django. Mechanizm ten działa w modelu *Challenge-Response* i składa się z następujących etapów technicznych:

1. **Generowanie wyzwania:** Serwer losuje ciąg znaków (np. 5 liter).
2. **Zapis w bazie danych:** Wylosowany ciąg nie jest wysyłany do klienta jawnym tekstem. Zostaje on zapisany w specjalnej tabeli systemowej `captcha_captchastore` wraz z:
 - Unikalnym kluczem (Hash Key).
 - Czasem wygaśnięcia (Expiration Time).
 - Poprawną odpowiedzią (Response).
3. **Renderowanie obrazu:** Przy użyciu biblioteki graficznej **Pillow (PIL)**, serwer generuje obraz rastrowy zawierający wylosowany tekst. Aby utrudnić odczytanie go przez algorytmy OCR (Optical Character Recognition), na obraz nakładane są filtry zniekształcające, szum oraz linie przecinające tekst.

W kodzie formularza (Rysunek 10) dodano pole `CaptchaField`, które automatycznie obsługuje ten proces.

```
try:
    from captcha.fields import CaptchaField

    HAS_CAPTCHA = True
except ImportError:
    HAS_CAPTCHA = False

3 usages
class SignupStep1Form(UserCreationForm):
    email = forms.EmailField(label="E-mail", required=True)

    if HAS_CAPTCHA:
        captcha = CaptchaField(label="Przepisz tekst z obrazka")
```

Rysunek 10: Implementacja pola `CaptchaField` w klasie formularza.

4.2 Proces walidacji po stronie serwera

Gdy użytkownik przesyła formularz, w żądaniu POST wysyłane są dwa parametry związane z CAPTCHA:

1. `captcha_0`: Ukryte pole zawierające hash klucza (identyfikator rekordu w bazie).
2. `captcha_1`: Tekst wpisany przez użytkownika.

Walidator po stronie serwera pobiera poprawną odpowiedź z bazy danych na podstawie klucza `captcha_0`. Następnie porównuje ją z wartością `captcha_1` (zazwyczaj ignorując wielkość liter). Jeśli wartości się zgadzają, a czas na rozwiązanie nie upłynął (domyślnie 5 minut, konfigurowalne przez `CAPTCHA_TIMEOUT`), walidacja kończy się sukcesem, a rekord jest usuwany z bazy, aby zapobiec atakom typu *Replay Attack*.

4.3 Interfejs użytkownika

W interfejsie użytkownika pojawia się wygenerowany obraz oraz pole tekstowe (Rysunek 11). Bez poprawnego rozwiązania zagadki, metoda `form.is_valid()` zwróci wartość `False`, blokując proces rejestracji.

Rejestracja

Nazwa użytkownika

slapinskaaaaa

E-mail

slapinskaaaa@gmail.com

Hasło

.....

Min. 8 znaków, nie może być zbyt podobne do danych konta. Musi zawierać co najmniej jeden znak specjalny.

Powtórz hasło

.....

Weryfikacja



MKGC

Utwórz konto

Masz już konto? [Zaloguj się](#)

Rysunek 11: Formularz rejestracji z widocznym, zaszumionym obrazem CAPTCHA.

4.4 Weryfikacja adresu e-mail (Double Opt-In)

Jako drugą linię obrony wdrożono weryfikację adresu e-mail. Nawet po przejściu CAPTCHA, konto pozostaje nieaktywne (`is_active=False`) do momentu potwierdzenia własności skrzynki pocztowej.

Proces ten przedstawiono na poniższych zrzutach ekranu:

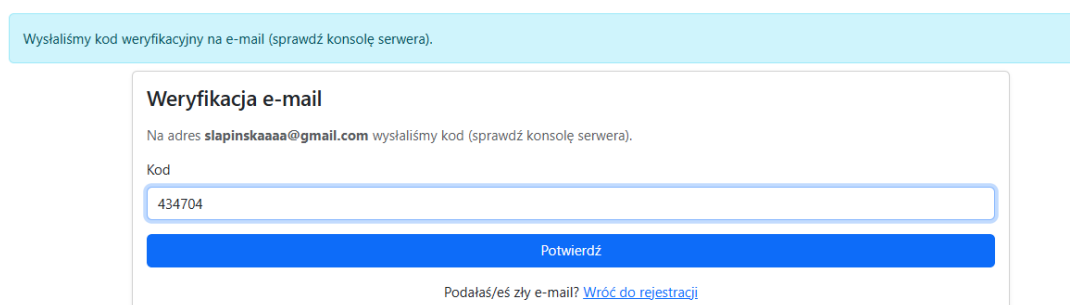
Wróć do rejestracji'." data-bbox="153 214 835 352"/>

Rysunek 12: Komunikat o konieczności weryfikacji po wstępnej rejestracji.

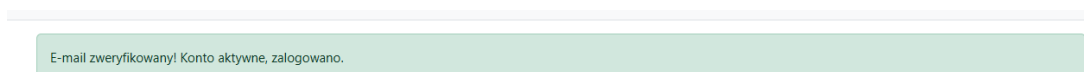
```
From: no-reply@travelly.local
To: slapinskaaaa@gmail.com
Date: Sun, 23 Nov 2025 22:18:20 -0000
Message-ID: <176393630053.3256.13691880156020831124@DESKTOP-316C4UE>

Twój kod to: 434704
-----
```

Rysunek 13: Wiadomość e-mail z kodem weryfikacyjnym generowanym losowo.



Rysunek 14: Wprowadzanie kodu aktywacyjnego.



Rysunek 15: Pomyślna aktywacja konta w systemie.

5 Ochrona przed atakami XSS (Cross-Site Scripting)

Atak Cross-Site Scripting (XSS) polega na wstrzyknięciu złośliwego skryptu (zazwyczaj JavaScript) do treści strony przeglądanej przez innych użytkowników. Jest to jedna z najczęstszych podatności aplikacji internetowych.

Jeśli atak się powiedzie, wstrzyknięty skrypt wykonuje się w przeglądarce ofiary z jej uprawnieniami. Konsekwencje takiego ataku mogą być krytyczne dla bezpieczeństwa systemu:

1. **Kradzież sesji (Session Hijacking):** Jest to najpoważniejsze zagrożenie. Skrypt może odczytać ciasteczko sesyjne (np. `sessionid`) i przesłać je do atakującego. Ponieważ serwer rozpoznaje użytkownika wyłącznie na podstawie tego ciasteczka, atakujący może "podszyć się" pod ofiarę i przejąć pełną kontrolę nad jej kontem bez znajomości loginu ani hasła.
2. **Nieautoryzowane akcje:** Skrypt może wykonywać operacje w imieniu zalogowanego użytkownika, np. zmienić hasło, wysłać wiadomości czy usunąć dane.
3. **Phishing i przekierowania:** Atakujący może podmienić treść strony (np. formularz logowania) lub przekierować użytkownika na fałszywą witrynę w celu wyłudzenia danych.

5.1 Mechanizm auto-escape w Django

Aby zapobiec powyższym scenariuszom, framework Django posiada wbudowany system ochrony. Szablony HTML domyślnie stosują mechanizm **auto-escape**.

Oznacza to, że wszelkie zmienne przekazywane z widoku do szablonu są traktowane przez silnik renderujący jako zwykły tekst, a nie kod wykonywalny. Znaki specjalne, które mogłyby posłużyć do konstrukcji tagów HTML, są automatycznie zamieniane na bezpieczne encje (tzw. escape characters):

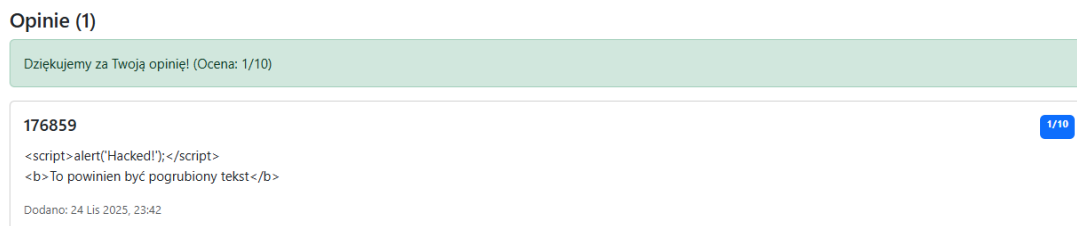
- `<` zamieniane na `<`;
- `>` zamieniane na `>`;
- `'` zamieniane na `'`;
- `"` zamieniane na `"`;
- `&` zamieniane na `&`;

Dzięki temu, nawet jeśli atakujący wprowadzi kod `<script>`, przeglądarka wyświetli go jako tekst, ale go nie wykona.

5.2 Test podatności (Weryfikacja)

W celu praktycznej weryfikacji tego zabezpieczenia przeprowadzono próbę ataku typu *Stored XSS*. W formularzu dodawania opinii, w polu treści wprowadzono złośliwy kod JavaScript: `<script>alert('XSS')</script>`.

Po zatwierdzeniu formularza i odświeżeniu strony, aplikacja wyświetliła wprowadzony ciąg znaków jako bezpieczny tekst. Okno dialogowe (funkcja `alert`) nie zostało uruchomione, co potwierdza skuteczność mechanizmu escape'owania (Rysunek 16).



Rysunek 16: Wynik próby ataku XSS – kod został zneutralizowany i wyświetlony jako tekst.

6 Bezpieczeństwo transmisji danych (HTTP vs HTTPS)

Ochrona kanału komunikacji między klientem a serwerem jest niezbędna, aby zapobiec podsłuchiowaniu (sniffing) oraz modyfikacji danych w locie (Man-in-the-Middle).

6.1 Analiza porównawcza protokołów

W poniższej tabeli zestawiono kluczowe różnice między standardowym protokołem HTTP a jego bezpieczną wersją HTTPS, którą wdrożono w ramach testów aplikacji.

Cecha	HTTP (Hypertext Transfer Protocol)	HTTPS (HTTP Secure)
Szyfrowanie	Brak (Plain Text). Dane przesyłane są jawnym tekstem, łatwym do przechwycenia.	TLS/SSL. Dane są szyfrowane algorytmami kryptograficznymi przed wysłaniem.
Uwierzytelnienie	Brak weryfikacji tożsamości serwera.	Certyfikat Cyfrowy. Potwierdza tożsamość serwera (zapobiega phishingowi).
Integralność danych	Brak ochrony przed modyfikacją danych w transporcie.	Sumy kontrolne. Gwarantują, że dane nie zostały zmienione w trakcie przesyłu.
Port komunikacyjny	Port 80	Port 443
Wskaźnik w przeglądarce	Ostrzeżenie "Niebezpieczona" lub brak ikony.	Ikona kłódki. Informacja o bezpiecznym połączeniu.

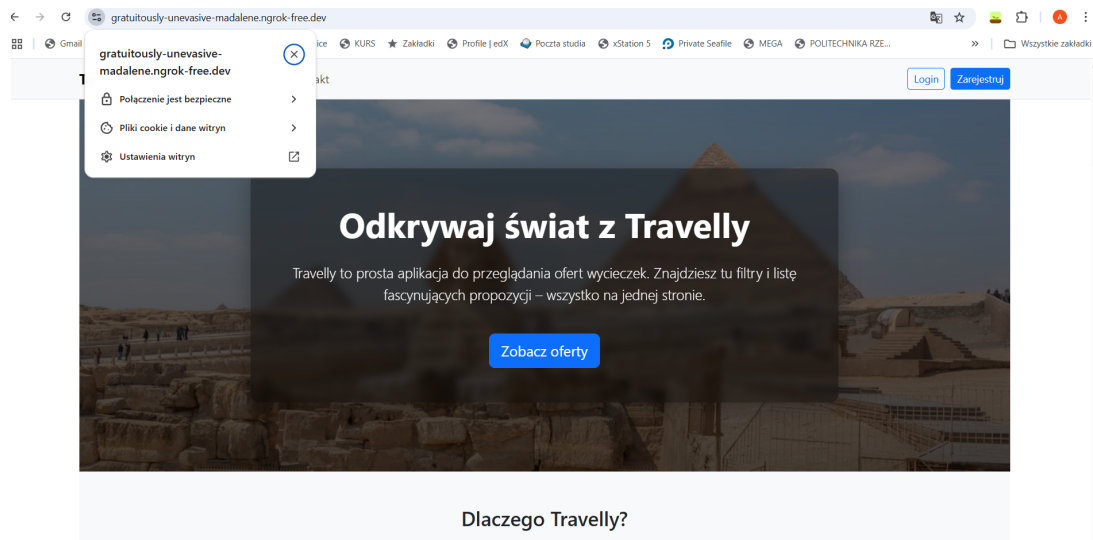
Tabela 1: Porównanie charakterystyki protokołów HTTP i HTTPS.

6.2 Weryfikacja szyfrowania (Tunelowanie SSL)

Choć środowisko deweloperskie domyślnie pracuje na protokole HTTP, w celu weryfikacji zachowania aplikacji w bezpiecznym środowisku przeprowadzono testy z użyciem tunelowania SSL (narzędzie **ngrok**).

Metoda ta pozwoliła na symulację warunków produkcyjnych i uzyskanie ważnego cer-

tyfikatu TLS. Jak widać na załączonym zrzucie ekranu (Rysunek 17), przeglądarka poprawnie zidentyfikowała połączenie jako bezpieczne.



Rysunek 17: Aplikacja działająca pod protokołem HTTPS (widoczna kłódka i komunikat "Połączenie jest bezpieczne").

Gwarantuje to użytkownikom:

1. **Poufność:** Hasła i dane osobowe są niemożliwe do odczytania przez osoby trzecie.
2. **Zaufanie:** Użytkownik ma pewność, że łączy się z właściwym serwerem "Travelly".

6.3 Konfiguracja Environment Hardening

W pliku `settings.py` zaimplementowano flagi bezpieczeństwa, które wymuszają korzystanie z szyfrowania w środowisku produkcyjnym (zostają aktywowane w trybie `DEBUG = False`). Obejmują one:

- `SESSION_COOKIE_SECURE = True` – Ciasteczko sesyjne jest przesyłane tylko po bezpiecznym łączu, co chroni przed kradzieżą sesji (Session Hijacking).
- `CSRF_COOKIE_SECURE = True` – Token chroniący formularze jest wysyłany wyłącznie kanałem szyfrowanym.
- `SECURE_SSL_REDIRECT = True` – Każda próba wejścia przez niezabezpieczone HTTP jest automatycznie przekierowywana na HTTPS.

Dzięki tej konfiguracji oraz pozytywnym wynikom testów tunelowania, aplikacja spełnia wymogi bezpieczeństwa transmisji danych.

7 Zarządzanie Sesją i Ochrona CSRF

7.1 Czym jest atak CSRF?

Cross-Site Request Forgery (CSRF) to atak, w którym złośliwa witryna wymusza na przeglądarce użytkownika wykonanie niechcianej akcji w zaufanym serwisie, w którym użytkownik jest aktualnie zalogowany. Dzieje się tak, ponieważ przeglądarki automatycznie dołączają ciasteczka sesyjne do każdego żądania wysyłanego do danej domeny. Bez dodatkowej ochrony serwer nie jest w stanie odróżnić, czy kliknięcie w przycisk "Usuń konto" pochodziło z woli użytkownika, czy zostało wywołane przez ukryty skrypt na innej stronie.

7.2 Ochrona CSRF w aplikacji

Aby zapobiec temu zagrożeniu, wdrożono mechanizm tokenów synchronizujących. Za obsługę odpowiada warstwa pośrednia (*Middleware*) włączona w konfiguracji `settings.py` (Rysunek 18). Sprawdza ona każde żądanie modyfikujące dane (POST, PUT, DELETE).



```
MIDDLEWARE = [  
    'django.middleware.security.SecurityMiddleware',  
    'django.contrib.sessions.middleware.SessionMiddleware',  
    'django.middleware.common.CommonMiddleware',  
    'django.middleware.csrf.CsrfViewMiddleware',  
    'django.contrib.auth.middleware.AuthenticationMiddleware',  
    'django.contrib.messages.middleware.MessageMiddleware',  
    'django.middleware.clickjacking.XFrameOptionsMiddleware',  
]
```

Rysunek 18: Aktywacja ochrony CSRF w settings.py (CsrfViewMiddleware).

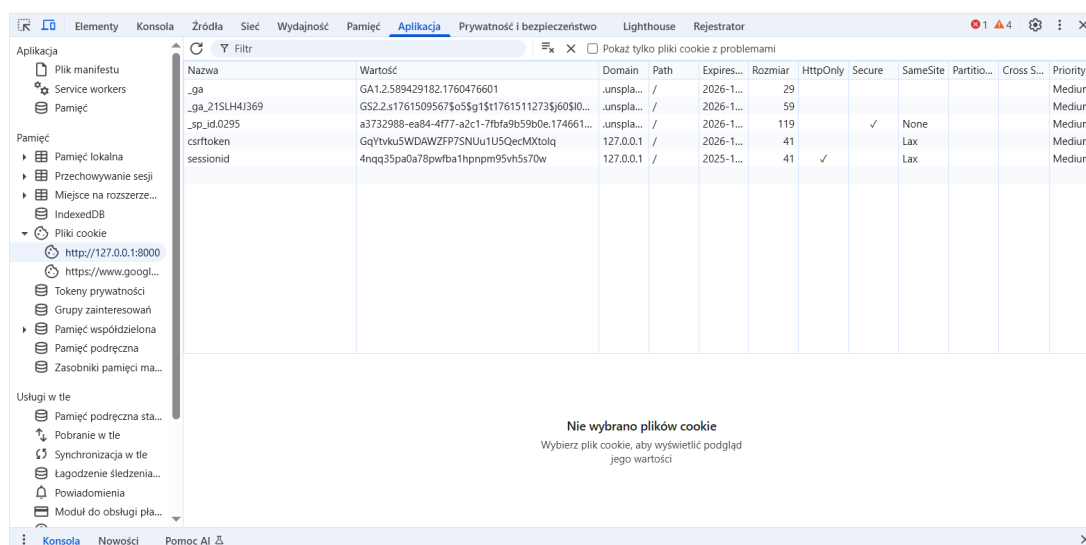
Każdy formularz w aplikacji musi zawierać unikalny, tajny token generowany przez serwer. Jest on wstrzykiwany do kodu HTML (tag `{% csrf_token %}`) jako ukryte pole (Rysunek 19). Jeśli żądanie przyjdzie bez tokena lub z nieprawidłowym tokenem, serwer odrzuci je (błąd 403 Forbidden).



Rysunek 19: Ukryty token CSRF widoczny w źródle strony HTML.

7.3 Bezpieczeństwo ciasteczek (HttpOnly)

Dodatkowo, aby zapobiec kradzieży identyfikatora sesji (np. przez atak XSS), wdrożono flagę **HttpOnly**. Sprawia ona, że skrypty JavaScript (np. `document.cookie`) nie mają dostępu do ciasteczka sesyjnego. Dostęp do niego ma tylko przeglądarka podczas komunikacji HTTP (Rysunek 20).



Rysunek 20: Analiza ciasteczek w narzędziach deweloperskich. Widoczna flaga HttpOnly.

8 Zabezpieczenia nagłówków HTTP i Infrastruktury

Oprócz ochrony danych wpisywanych w formularze, kluczowe jest również zabezpieczenie samej komunikacji przeglądarki z serwerem. W tym celu skonfigurowano odpowiednie nagłówki HTTP.

8.1 Ochrona przed Clickjackingiem (Niewidzialne ramki)

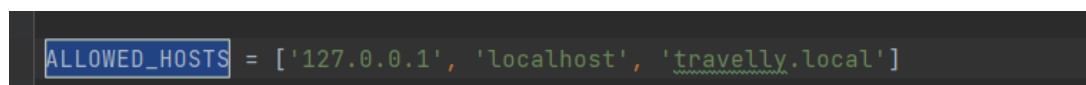
Clickjacking to podstępny atak, w którym haker umieszcza naszą stronę w niewidocznej ramce (`<iframe>`) na swojej złośliwej witrynie. Użytkownik myśli, że klika np. w przycisk "Wygraj nagrodę", a w rzeczywistości klika w niewidoczny pod spodem przycisk "Usuń konto" w naszej aplikacji.

Aby temu zapobiec, w pliku `settings.py` aktywowano `XFrameOptionsMiddleware`. Dodaje ono do każdego żądania nagłówek `X-Frame-Options` z wartością **DENY** lub **SAMEORIGIN**. Jest to instrukcja dla przeglądarki: "Nigdy nie pozwalaj na wyświetlanie tej strony w ramce na obcych witrynach".

8.2 Ochrona przed podszywaniem się (Host Header Injection)

Serwer musi wiedzieć, pod jakimi adresami (domenami) ma prawo odpowiadać. Bez tej weryfikacji, atakujący mógłby wysłać żądanie ze zmodyfikowanym nagłówkiem "Host", co mogłoby zmylić aplikację (np. generując linki do resetu hasła prowadzące do domeny haker).

W pliku `settings.py` skonfigurowano "białą listę" bezpiecznych adresów w zmiennej `ALLOWED_HOSTS` (Rysunek 21). Jeśli serwer otrzyma żądanie skierowane na inny adres (np. adres IP zamiast nazwy domeny), natychmiast je odrzuci, zwracając błąd "400 Bad Request".



```
ALLOWED_HOSTS = ['127.0.0.1', 'localhost', 'travelly.local']
```

Rysunek 21: Konfiguracja dozwolonych hostów w pliku `settings.py` – serwer obsłuży tylko te adresy.

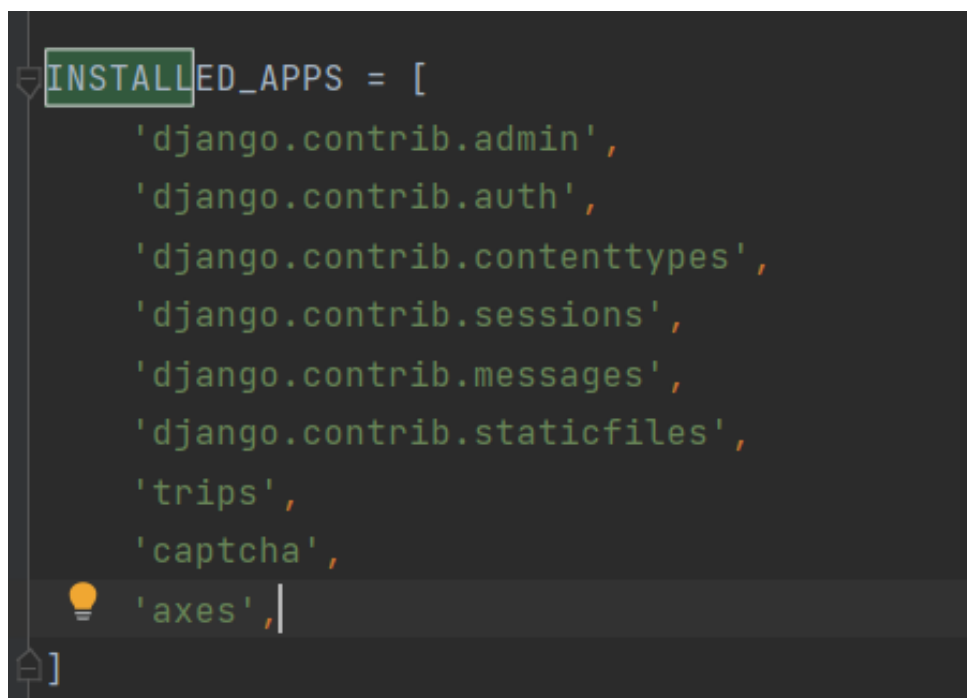
9 Aktywna ochrona przed atakami Brute-Force

Ataki siłowe (Brute-Force) polegają na próbie odgadnięcia hasła poprzez sprawdzanie tysięcy kombinacji w krótkim czasie. Standardowy system logowania nie posiada limitów prób, co teoretycznie pozwalałoby hakerowi próbować w nieskończoność. Aby zamknąć tę furtkę, wdrożono bibliotekę ****django-axes****.

9.1 Instalacja i podłączenie 'ochroniarza'

Biblioteka ta działa jak strażnik, który monitoruje każdą próbę logowania. Aby działała poprawnie, musieliśmy wprowadzić trzy zmiany w pliku **settings.py**:

1. Dodanie **axes** do listy zainstalowanych aplikacji (Rysunek 22).
2. Włączenie **AxesStandaloneBackend**, co pozwala systemowi "pamiętać" nieudane próby logowania.
3. Dodanie warstwy pośredniej **AxesMiddleware** (Rysunek 23), która sprawdza każdego użytkownika, zanim jeszcze zobaczy formularz logowania.



```
INSTALLED_APPS = [  
    'django.contrib.admin',  
    'django.contrib.auth',  
    'django.contrib.contenttypes',  
    'django.contrib.sessions',  
    'django.contrib.messages',  
    'django.contrib.staticfiles',  
    'trips',  
    'captcha',  
    'axes',  
]
```

Rysunek 22: Rejestracja modułu axes w konfiguracji projektu.


```
MIDDLEWARE = [
    'django.middleware.security.SecurityMiddleware',
    'django.contrib.sessions.middleware.SessionMiddleware',
    'django.middleware.common.CommonMiddleware',
    'django.middleware.csrf.CsrfViewMiddleware',
    'django.contrib.auth.middleware.AuthenticationMiddleware',
    'django.contrib.messages.middleware.MessageMiddleware',
    'django.middleware.clickjacking.XFrameOptionsMiddleware',
    'axes.middleware.AxesMiddleware',
]
```

Rysunek 23: Aktywacja Middleware, który monitoruje ruch przychodzący.

9.2 Konfiguracja polityki bezpieczeństwa

Ustaliliśmy surowe, ale sprawiedliwe reguły gry (Rysunek 24):

- **Limit błędów:** 3 próby. Po trzecim błędnym hasle system uznaje, że następuje atak.
- **Metoda blokady:** Blokujemy kombinację Użytkownik + Adres IP. Dzięki temu, jeśli ktoś próbuje włamać się na konto "Admin", prawdziwy administrator logujący się z innego komputera nie zostanie zablokowany.
- **Czas kary:** Blokada trwa określony czas (Cool-off period), co czyni atak siłowy nieopłacalnym czasowo.

```
# Konfiguracja AXES (Brute-Force Protection)
AXES_FAILURE_LIMIT = 3 # Blokada po 3 błędach
AXES_COOLOFF_TIME = 1 # Blokada na 1 godzinę
AXES_LOCKOUT_TEMPLATE = 'lockout.html' #
AXES_RESET_ON_SUCCESS = True # Reset licznika jak się uda zalogować
```

Rysunek 24: Definicja limitów prób logowania w settings.py.

Dla lepszego doświadczenia użytkownika (UX), zamiast standardowego błędu serwera, przygotowaliśmy specjalną stronę informacyjną `lockout.html` (Rysunek 25).

```
{% extends "base.html" %}

{% block content %}
<div class="container text-center mt-5" style="min-height: 50vh; display: flex; align-items: center; justify-content: center;">
  <div class="alert alert-danger shadow-lg p-5">
    <h1 class="display-1">🚫</h1>
    <h2 class="display-4 fw-bold">Konto zablokowane</h2>
    <hr>
    <p class="lead">Wykryto zbyt wiele nieudanych prób logowania.</p>
    <p>Ze względów bezpieczeństwa dostęp został tymczasowo zablokowany.<br>Spróbuj ponownie za kilka minut.</p>
    <a href="/" class="btn btn-outline-danger mt-3">Wróć na stronę główną</a>
  </div>
</div>
{% endblock %}
```

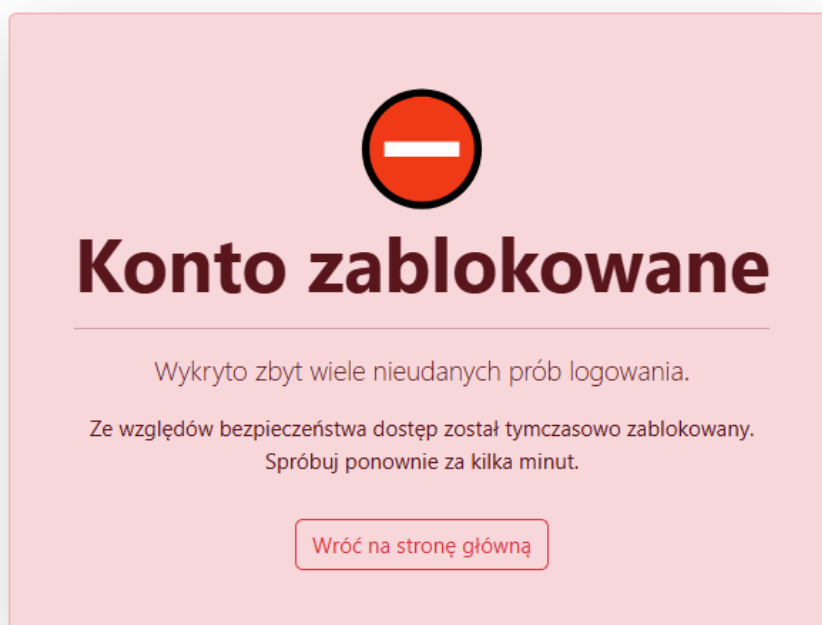
Rysunek 25: Kod szablonu strony wyświetlanej zablokowanemu użytkownikowi.

9.3 Weryfikacja skuteczności (Symulacja ataku)

W celu sprawdzenia systemu celowo wpisywaliśmy błędne hasła.

- Próby 1-2: System wyświetlał standardowy komunikat "Błędne dane".
- Próba 3: Dostęp został zablokowany.
- Próba 4: Mimo wpisania **poprawnego** hasła, system odmówił dostępu i wyświetlił ekran blokady (Rysunek 26).

Dowodzi to, że mechanizm skutecznie chroni przed automatami zgadującymi hasła.



Rysunek 26: Efekt działania zabezpieczenia – konto zablokowane po przekroczeniu limitu.

10 Monitorowanie i Rejestrowanie Zdarzeń (Audyt)

System posiada wbudowany moduł audytowy, który rejestruje kluczowe zdarzenia w systemie, zapewniając rozliczalność działań użytkowników (ang. *Accountability*).

10.1 Logi systemowe i historia zmian

W panelu administracyjnym automatycznie tworzone są wpisy rejestrujące operacje na danych (CRUD). Mechanizm ten pozwala administratorowi na śledzenie zmian i szybkie wykrycie potencjalnie nieautoryzowanych działań, np. zmiany uprawnień przez innego administratora (Rysunek 27).

Historia zmian: admin

DATA/CZAS	UZYTKOWNIK	AKCJA
30 listopada 2025 19:54	admin (Administrator)	Zmodyfikowano First name.

Rysunek 27: Dziennik zdarzeń (historia operacji) w panelu administracyjnym.

11 Szczegółowa analiza konfiguracji (settings.py)

Plik `settings.py` jest centrum sterowania aplikacją Django. Poniżej przedstawiono analizę jego zawartości podzieloną na sekcje, zgodnie z załączonymi fragmentami kodu.

11.1 Sekcja 1: Podstawowe ustawienia środowiska

Na pierwszym fragmencie (Rysunek 28) widoczne są kluczowe zmienne inicjalizujące projekt.

```
from pathlib import Path
BASE_DIR = Path(__file__).resolve().parent.parent

|

# Quick-start development settings - unsuitable for production
# See https://docs.djangoproject.com/en/5.2/howto/deployment/checklist/

# SECURITY WARNING: keep the secret key used in production secret!
SECRET_KEY = 'django-insecure-ykt0+7a_vreywk!@!7aum31jj2mo_o1=b6o&ep*3m%o=+@8@ds'

# SECURITY WARNING: don't run with debug turned on in production!
DEBUG = True

ALLOWED_HOSTS = ['127.0.0.1', 'localhost', 'travelly.local']

# Application definition

INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'trips',
]
```

Rysunek 28: Zmienne środowiskowe, tryb debugowania i zainstalowane aplikacje.

Opis kluczowych pól:

- **SECRET_KEY:** Kryptograficzny klucz używany do podpisywania sesji i tokenów. Jego ujawnienie (co jest widoczne tutaj w celach edukacyjnych) w środowisku produkcyjnym pozwoliłoby atakującemu na fałszowanie tożsamości użytkowników.
- **DEBUG = True:** Tryb deweloperski. Zapewnia szczegółowe raporty o błędach, ale musi zostać bezwzględnie wyłączony na produkcji, aby nie ujawniać struktury kodu i danych.

- `ALLOWED_HOSTS`: Lista domen, które serwer ma prawo obsłużyć. Chroni to przed atakami typu *Host Header Injection*.
- `INSTALLED_APPS`: Lista aktywnych modułów. Widoczne są tu domyślne moduły Django (`admin`, `auth`, `sessions`).

11.2 Sekcja 2: Aplikacje zewnętrzne i Middleware

Drugi fragment (Rysunek 29) przedstawia dodatkowe moduły bezpieczeństwa oraz warstwę pośrednią.

```
'trips',
'captcha',
'axes',
]

MIDDLEWARE = [
    'django.middleware.security.SecurityMiddleware',
    'django.contrib.sessions.middleware.SessionMiddleware',
    'django.middleware.common.CommonMiddleware',
    'django.middleware.csrf.CsrfViewMiddleware',
    'django.contrib.auth.middleware.AuthenticationMiddleware',
    'django.contrib.messages.middleware.MessageMiddleware',
    'django.middleware.clickjacking.XFrameOptionsMiddleware',
    'axes.middleware.AxesMiddleware',
]

ROOT_URLCONF = 'travelly.urls'

TEMPLATES = [
    {
        'BACKEND': 'django.template.backends.django.DjangoTemplates',
        'DIRS': [BASE_DIR / 'templates'],
        'APP_DIRS': True,
        'OPTIONS': {
            'context_processors': [
                'django.template.context_processors.debug',
                'django.template.context_processors.request',
                'django.contrib.auth.context_processors.auth',
```

Rysunek 29: Rejestracja bibliotek bezpieczeństwa i konfiguracja Middleware.

Analiza konfiguracji:

- **Dodatkowe aplikacje:** Widać zarejestrowane biblioteki: `'captcha'` (ochrona przed botami) oraz `'axes'` (ochrona przed Brute-Force).
- **MIDDLEWARE:** To lista "filtrów", przez które przechodzi każde żądanie. Kolejność jest krytyczna:
 - `SecurityMiddleware`: Obsługuje nagłówki bezpieczeństwa.

- SessionMiddleware: Zarządza ciasteczkami sesyjnymi.
- CsrfViewMiddleware: Chroni przed atakami CSRF.
- AxesMiddleware: Monitoruje próby logowania (musi być na końcu listy, aby analizować w pełni przetworzone żądanie).

11.3 Sekcja 3: Baza danych i walidatory (cz. 1)

Trzeci zrzut (Rysunek 30) pokazuje konfigurację bazy danych oraz początek sekcji walidacji haseł.

```

    'django.contrib.messages.context_processors.messages',
],
},
},
]

WSGI_APPLICATION = 'travelly.wsgi.application'

# Database
# https://docs.djangoproject.com/en/5.2/ref/settings/#databases

DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.sqlite3',
        'NAME': BASE_DIR / 'db.sqlite3',
    }
}

# Password validation
# https://docs.djangoproject.com/en/5.2/ref/settings/#auth-password-validators

AUTH_PASSWORD_VALIDATORS = [
    {"NAME": "django.contrib.auth.password_validation.UserAttributeSimilarityValidator"},
    {"NAME": "django.contrib.auth.password_validation.MinimumLengthValidator"}
]
```

Rysunek 30: Konfiguracja bazy danych SQLite i pierwszy walidator haseł.

Opis pól:

- DATABASES: Aplikacja korzysta z domyślnej bazy `sqlite3`, która jest wystarczająca dla środowiska deweloperskiego.
- UserAttributeSimilarityValidator: Pierwszy z walidatorów. Zapobiega ustawieniu hasła, które jest zbyt podobne do danych użytkownika (np. hasło "admin123" dla użytkownika "admin").

11.4 Sekcja 4: Zaawansowana walidacja haseł i lokalizacja

Czwarty fragment (Rysunek 31) zawiera kluczowe reguły wymuszające siłę haseł.

```

AUTH_PASSWORD_VALIDATORS = [
    {"NAME": "django.contrib.auth.password_validation.UserAttributeSimilarityValidator"},
    {"NAME": "django.contrib.auth.password_validation.MinimumLengthValidator",
      "OPTIONS": {"min_length": 10}},
    {"NAME": "django.contrib.auth.password_validation.CommonPasswordValidator"},
    {"NAME": "django.contrib.auth.password_validation.NumericPasswordValidator"},
    {"NAME": "trips.validators.ComplexityValidator"},
]

# Internationalization
# https://docs.djangoproject.com/en/5.2/topics/i18n/

LANGUAGE_CODE = 'pl-pl'

TIME_ZONE = 'Europe/Warsaw'

USE_I18N = True

USE_TZ = True

# Static files (CSS, JavaScript, Images)
# https://docs.djangoproject.com/en/5.2/howto/static-files/

STATIC_URL = 'static/'

```

Rysunek 31: Lista walidatorów haseł, w tym autorski ComplexityValidator.

Zastosowane walidatory:

- `MinimumLengthValidator`: Wymusza minimalną długość hasła (skonfigurowano na 10 znaków).
- `CommonPasswordValidator`: Blokuje hasła z listy najpopularniejszych (np. "123456", "password").
- `NumericPasswordValidator`: Zapobiega hasłom składającym się z samych cyfr.
- `trips.validators.ComplexityValidator`: Autorski walidator napisany na potrzeby projektu, który wymusza znaki specjalne i duże litery.

11.5 Sekcja 5: Pliki statyczne i Backendy Uwierzytelniania

Piąty zrzut (Rysunek 32) dotyczy obsługi plików oraz mechanizmu logowania.

```

STATIC_ROOT = BASE_DIR / 'staticfiles'
|
STATICFILES_DIRS = [
    BASE_DIR / "static",
]

# Default primary key field type
# https://docs.djangoproject.com/en/5.2/ref/settings/#default-auto-field

DEFAULT_AUTO_FIELD = 'django.db.models.BigAutoField'

LOGIN_REDIRECT_URL = '/'

LOGOUT_REDIRECT_URL = '/'

# Wiadomości e-mail w DEV leca do konsoli (terminal)
EMAIL_BACKEND = "django.core.mail.backends.console.EmailBackend"
DEFAULT_FROM_EMAIL = "no-reply@travelly.local"

# Gdzie trafiaamy po logowaniu/wylogowaniu
LOGIN_REDIRECT_URL = "/"
LOGOUT_REDIRECT_URL = "/"
LOGIN_URL = "/accounts/login/"

AUTHENTICATION_BACKENDS = [
    'axes.backends.AxesStandaloneBackend',

```

Rysunek 32: Konfiguracja backendów uwierzytelniania dla modułu Axes.

Kluczowy element bezpieczeństwa:

- **AUTHENTICATION_BACKENDS:** Domyślnie Django używa tylko `ModelBackend`. Tutaj dodano `axes.backends.AxesStandaloneBackend`. Jest to niezbędne, aby biblioteka `django-axes` mogła przechwytywać sygnały logowania (sukces/porażka) i na ich podstawie blokować ataki siłowe.

11.6 Sekcja 6: Ochrona Brute-Force i Environment Hardening

Ostatni fragment (Rysunek 33) zawiera konfigurację limitów logowania oraz zabezpieczenia produkcyjne.


```

    'django.contrib.auth.backends.ModelBackend',
]

# Konfiguracja AXES (Brute-Force Protection)
AXES_FAILURE_LIMIT = 3 # Blokada po 3 błędach
AXES_COOLOFF_TIME = 1 # Blokada na 1 godzinę
AXES_LOCKOUT_TEMPLATE = 'lockout.html' #
AXES_RESET_ON_SUCCESS = True # Reset licznika jak się uda zalogować

# =====
# KONFIGURACJA BEZPIECZEŃSTWA (ENVIRONMENT HARDENING)
# =====

# Te ustawienia aktywują się tylko na produkcji (gdy DEBUG = False)
if not DEBUG:
    # Wymuszenie HTTPS
    SECURE_SSL_REDIRECT = True

    # Bezpieczne ciasteczka (tylko po HTTPS)
    SESSION_COOKIE_SECURE = True
    CSRF_COOKIE_SECURE = True

    # HSTS (HTTP Strict Transport Security) - wymusza HTTPS przez rok
    SECURE_HSTS_SECONDS = 31536000
    SECURE_HSTS_INCLUDE_SUBDOMAINS = True
    SECURE_HSTS_PRELOAD = True

```

Rysunek 33: Polityka blokad (Axes) i utwardzanie środowiska produkcyjnego.

Szczegółowy opis:

- **Konfiguracja AXES:**

- AXES_FAILURE_LIMIT = 3: Konto/IP jest blokowane po 3 błędnych próbach.
- AXES_COOLOFF_TIME = 1: Czas blokady wynosi 1 godzinę.
- AXES_RESET_ON_SUCCESS: Poprawne logowanie zeruje licznik błędów.

- **Environment Hardening (if not DEBUG):** Sekcja ta aktywuje się tylko po wyłączeniu trybu deweloperskiego.

- SECURE_SSL_REDIRECT: Wymusza szyfrowanie HTTPS dla każdego połączenia.
- SESSION_COOKIE_SECURE: Ciasteczko sesji jest wysyłane tylko kanałem szyfrowanym.
- SECURE_HSTS_SECONDS: Włącza mechanizm HSTS, zmuszając przeglądarki do pamiętania, że ta strona wymaga HTTPS (ochrona przed atakiem SSL Striping).

12 Analiza Ryzyka i Podsumowanie

12.1 Macierz Analizy Ryzyka

W celu oceny skuteczności wdrożonych zabezpieczeń przeprowadzono analizę ryzyka. Poniższa tabela przedstawia, jak zaimplementowane mechanizmy wpłynęły na redukcję zagrożeń.

Zagrożenie	Ryzyko Pierwotne	Zastosowane Zabezpieczenie	Ryzyko Szczatkowe
Brute-force (Hasła)	WYSOKIE	PBKDF2, django-axes (Blokada IP)	NISKIE
SQL Injection	KRYTYCZNE	Django ORM + Autorska walidacja	BARDZO NISKIE
XSS (Skrypty)	WYSOKIE	Auto-escape, HttpOnly Cookie	NISKIE
Clickjacking	ŚREDNIE	X-Frame-Options Middleware	NISKIE
Host Header Inj.	WYSOKIE	Walidacja ALLOWED_HOSTS	BARDZO NISKIE
Kradzież sesji	WYSOKIE	Tokeny CSRF + Middleware	BARDZO NISKIE

Tabela 2: Macierz redukcji ryzyka w systemie "Travelly".

12.2 Wnioski

Aplikacja "Travelly" została zabezpieczona zgodnie z aktualnymi standardami bezpieczeństwa webowego. Zastosowanie modelu *Defense in Depth* (obrona w głąb) – łączącego mechanizmy wbudowane w framework (ORM, CSRF, Secure Cookies) z dodatkowymi bibliotekami (django-axes, django-simple-captcha) i autorskimi walidatorami – pozwoliło na stworzenie systemu odpornego na kluczowe podatności z listy OWASP Top 10. Plik `settings.py` został zweryfikowany pod kątem bezpiecznej konfiguracji produkcyjnej.